Aula 5 — KMP: Kotlin Multiplatform + BFF

"Uma base de código, três plataformas"

Prof. Jackson Smith Moisés Matias
PUC Minas IEC

Recap da Aula 4

- PWA: Web App Manifest + Service Worker lifecycle
- Cache strategies: CacheFirst, NetworkFirst, StaleWhileRevalidate
- Workbox: abstração declarativa de SW
- Background Sync + IndexedDB
- **Atividade 4** (PWA + TMDB) — prazo no Canvas; dúvidas, fale comigo

O problema: 3 meses, time pequeno, 3 plataformas

> *"Recebemos aporte. Precisamos lançar o app pra* **Android, iOS e Web em 3 meses** *— mas o orçamento só dá pra contratar um time pequeno. A diretoria quer saber: nativo (3 times) ou multiplataforma (1 time, 1 base de código)? E se for multiplataforma — qual?"*

Hoje respondemos com **KMP**: 1 base de lógica compartilhada, 3 alvos de plataforma. A pergunta que interessa não é "como escrever Kotlin" — é **onde essa promessa aperta na prática**.

Agenda da aula

Bloco 1 (90min) — KMP foundations: código compartilhado entre Android, iOS e Web
→ KotlinProject · setup toolchain · arquitetura shared · Ktor + TMDB · `expect` / `actual`

Intervalo (30min)

Bloco 2 (60min) — KMP na prática + Trabalho Final
→ **Atividade 5 ao vivo** (exercício KMP) · **Projeto Final (30 pts)**: 5 temas · 3 frameworks · rubrica · logística
→ BFF/GraphQL só o conceito (sem demo ao vivo)

Objetivos de aprendizagem

- Compartilhar lógica de negócio entre Android, iOS e Web com Kotlin
- Integrar HTTP com Ktor + kotlinx.serialization + TMDB API
- Entender `expect` / `actual` para código platform-specific
- Entender padrão Backend-for-Frontend — um servidor, múltiplos clientes
- Comparar REST (KMP/PWA) vs GraphQL — conceito e quando usar

KotlinProject — como foi criado

Projeto gerado pelo **KMP wizard oficial** da JetBrains:

> **kmp.jetbrains.com**
> Config usada: Android ✓ · iOS ✓ · iOS UI = **Compose** · Include Tests ✓

Essa URL reproduz exatamente o starter. Qualquer aluno pode gerar o mesmo projeto do zero no wizard — é o ponto de partida oficial pra projetos KMP novos.

O que vamos rodar hoje — KotlinProject

Três apps, **um módulo compartilhado** (`shared` — Compose Multiplatform):

App	Módulo	Plataforma
`androidApp`	`:androidApp`	Android
`iosApp`	`iosApp/`	iOS (Xcode project)
`webApp`	`:webApp`	Web (JS / Wasm)

Targets declarados em `shared/build.gradle.kts`:
`androidTarget`, `iosArm64`, `iosSimulatorArm64`, `js`, `wasmJs`

Toolchain verificado — versões exatas

Ferramenta	Versão
Android Studio	**Quail 2026.1.1 Patch 2**
Gradle wrapper	9.1.0
Kotlin	2.4.0
AGP	9.0.1
Compose Multiplatform	1.11.1
JDK (terminal)	OpenJDK 17.0.15
JDK (Studio)	bundled OpenJDK 21.0.10
Xcode	26.1.1

> Projeto **não carrega** no Studio < Quail 2026.1.1. Atualizar se necessário.

Gotchas: Apple Silicon usa `iosSimulatorArm64` (`iosX64` não declarado → simuladores Intel não buildam) · config cache instável → `./gradlew --no-configuration-cache <task>` · nunca commitar `local.properties` (caminho de SDK é por máquina).

Setup — pré-requisitos

```
# 1. Verificar Xcode CLI
xcode-select -p
# → /Applications/Xcode.app/Contents/Developer   ✓
```

- **macOS** + Homebrew (`/opt/homebrew/bin/brew`)
- **Android Studio Quail 2026.1.1+** — versão exata importa
- **Xcode 26.x** com command-line tools selecionados
- `local.properties` criado automaticamente pelo Studio:

  ```
  sdk.dir=/Users/<seu-user>/Library/Android/sdk
  ```

  > Não commitar — é caminho de máquina.

Setup — plugin KMP no Android Studio

> **A parte mais traiçoeira.** A configuração `iosApp` só aparece depois que o plugin carrega.

Passo a passo:

1. **Settings → Plugins → Marketplace** → buscar **Kotlin Multiplatform** (JetBrains s.r.o.) → Install
2. Se aparecer erro **`Requires plugin 'com.intellij.marketplace' to be installed`** + badge vermelho:
   - Marketplace → buscar **JetBrains Marketplace Licensing** → Install
3. Reiniciar o Studio
4. Dropdown de run config agora mostra **`iosApp`** ao lado de `androidApp` ✓

> `kdoctor` pode reportar "KMM plugin not installed" — é falso positivo (procura o id legado 14936). Ignorar.

Verificar ambiente — kdoctor

```
brew install kdoctor
kdoctor            # ou /opt/homebrew/bin/kdoctor
```

Esperado: OS ✓ · Java ✓ · Xcode ✓

2 warnings seguros para ignorar:

- *"Android Studio: KMM plugin not installed"* — kdoctor 1.1.0 checa id legado; nosso plugin está ok
- *"CocoaPods not configured"* — este projeto não tem `Podfile`; só necessário com libs CocoaPods

iOS Simulator não existe? kdoctor avisa *"Couldn't find available Apple Simulators"* — resolve com:

```
xcrun simctl list devices available   # checar o que já existe
xcrun simctl create "iPhone 16" \
  com.apple.CoreSimulator.SimDeviceType.iPhone-16 \
  com.apple.CoreSimulator.SimRuntime.iOS-26-1
```

Rodando os 3 targets

Android

- Dropdown → `androidApp` + emulador/device → ▶️

iOS

- Dropdown → `iosApp` + `iPhone 16` → ▶️
- Fallback (sem plugin):

```
open iosApp/iosApp.xcodeproj   # Run pelo Xcode
```

Web

```
./gradlew :webApp:jsBrowserDevelopmentRun       # JS
./gradlew :webApp:wasmJsBrowserDevelopmentRun   # Wasm
```

Kotlin Multiplatform (KMP) — Stack & Forças

> É a opção que testamos hoje contra o problema do slide anterior: 1 base de lógica, 3 alvos.

Stack:

- **Kotlin** linguagem em Android, iOS via Kotlin/Native, JS, JVM, Linux, Windows
- `expect` / `actual` — contratos compartilhados, implementações por plataforma
- **Compose Multiplatform** — UI compartilhada (em maturação)
- **Ktor** — HTTP client
- **kotlinx.serialization** — JSON
- **BuildKonfig** — variáveis de ambiente/config compartilhadas (nosso `tmdb.token` do `local.properties` vira `BuildKonfig.TMDB_TOKEN` em todas as plataformas)

Forças:

- Compartilha **regra de negócio**, não UI (na maioria dos casos)
- Excelente integração com Android nativo
- iOS via Swift interop

O gargalo real do KMP

CASH APP — Square (hoje Block, Inc.)

30 milhões de usuários ativos/mês

App de pagamentos P2P, lançado em 2013 com 4 engenheiros mobile · adotou KMP em 2018 · 50 engenheiros mobile (iOS+Android) no case study JetBrains de 2021

Times de **network, investing e growth** rodam parte da lógica de negócio em KMP em produção (fonte: case study oficial JetBrains/kotlinlang.org). Bateram em 2 paredes documentadas:

- **iOS interop**: debugar Kotlin compartilhado a partir do Xcode/Swift é mais lento — breakpoint não "conversa" tão bem quanto no lado Android
- **Compose Multiplatform em iOS**: mais jovem que no Android — bugs de renderização que não existem do lado Android aparecem só no iOS

> Isso **não** é motivo pra descartar KMP — Square manteve em produção. É o **trade-off real** que vocês assinam: lógica compartilhada, debugging não-trivial em troca. **Lição do Bloco 1**: KMP compartilha regra de negócio, não elimina trabalho de plataforma.

Kotlin — sintaxe em 1 slide

```kotlin
val nome = "Ana"            // imutável (≈ const)
var contador = 0            // mutável
fun saudar(n: String) = println("Oi, $n")    // função

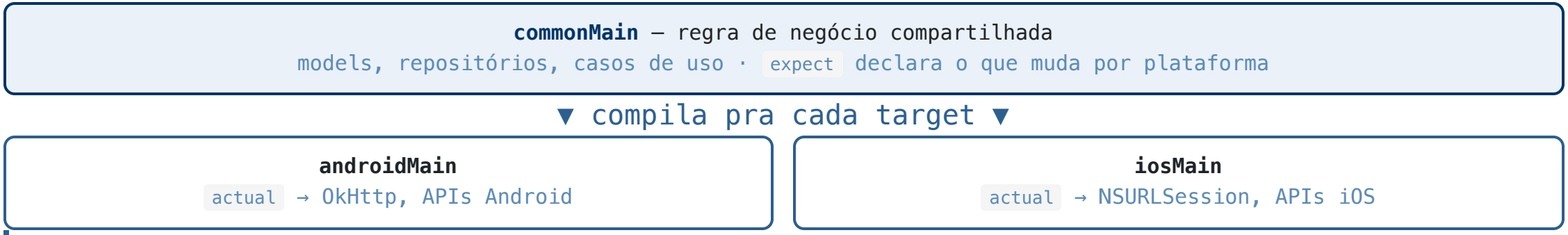
data class Filme(val titulo: String, val nota: Double)   // equals/copy/toString grátis

val apelido: String? = null          // ? = pode ser null
val tam = apelido?.length ?: 0       // safe-call (?.) + elvis (?:)
listOf(1, 2, 3).filter { it > 1 }    // lambda: it = o item
```

> `val` / `var` · `data class` = seu modelo · `?` / `?:` = null-safety · `{ it }` = lambda. É a linguagem do **"shared"** do KMP.

KMP — arquitetura (shared + targets)

> Resolve o gargalo de cima: lógica sensível (pagamento, favoritos) fica em `commonMain`, só o que exige API nativa vira `expect` / `actual` — o time de iOS debuga menos coisa nova, não zero.



> Compartilha **lógica**, não UI. O que é específico de plataforma fica nos `actual`.

Código por plataforma — você JÁ resolveu esse problema 2x

O dilema é sempre o mesmo: **uma chamada no código compartilhado, implementação diferente por SO.** Cada framework que estudamos tem uma resposta:

Framework	Mecanismo	Quando resolve	Onde você usou
React Native	`Platform.OS === 'ios'` · arquivos `.ios.tsx` / `.android.tsx`	runtime / bundler	Aula 2
Flutter	`MethodChannel` + `Platform.isIOS`	runtime	Atividade 3 (native module)
KMP	**`expect` / `actual`**	**compilação**	Atividade 5 (hoje)

A diferença do KMP: é **recurso da linguagem, checado pelo compilador** — `expect` declara o contrato no `commonMain`; cada target (`androidMain`, `iosMain`) é **obrigado** a ter o `actual`. Esqueceu uma plataforma? **Não compila.** No RN/Flutter, o esquecimento explode em runtime, na mão do usuário.

KMP — `expect` / `actual`

```
// commonMain
expect class HttpClient {
    suspend fun get(url: String): String
}

// androidMain
actual class HttpClient {
    actual suspend fun get(url: String): String {
        // OkHttp implementation
    }
}

// iosMain
actual class HttpClient {
    actual suspend fun get(url: String): String {
        // NSURLSession implementation
    }
}
```

`expect` declara o contrato. `actual` implementa por plataforma.

🌐 Hands-on guiado — Kotlin (Playground)

play.kotlinlang.org (zero instalação). Escreva a **"shared logic" dos favoritos** — a mesma do `favoritesStore` do app:

```kotlin
data class Movie(val id: Int, val title: String, val rating: Double)   // 1. modelo

fun favorites(movies: List<Movie>, favIds: Set<Int>) =                 // 2. função pura
  movies.filter { it.id in favIds }

fun main() {
  val all = listOf(
    Movie(1, "Matrix", 8.7),
    Movie(2, "Inception", 8.8),
    Movie(3, "Tenet", 7.4),
  )
  val favs = favorites(all, setOf(1, 3))                               // 3. rode!
  println(favs.map { it.title })   // → [Matrix, Tenet]
}
```

> (1) `data class` = modelo · (2) função pura de filtro · (3) rode. **Essa lógica o KMP compartilha** entre Android e iOS — só o que toca plataforma (rede, sensores) vira `expect` / `actual`.

☕ Intervalo — 30min

Volta às :___

Aproveita pra:

- Esticar perna
- Pegar token TMDB se não tiver: developer.themoviedb.org → API → Bearer
- Android Studio aberto com o projeto KMP carregado

> Próximo bloco: BFF/GraphQL + Atividade 5.

O problema que o BFF resolve (e o que não resolve)

> *"Nosso catálogo vive em* **5 microsserviços** *(filmes, elenco, avaliações, recomendações, preços). Servimos* **3 clientes** *(Android, iOS, site) e cada tela pede uma combinação diferente desses 5. Já temos* **23 endpoints REST** *sob medida — um por combinação tela×cliente — e ninguém lembra qual serve o quê. Toda mudança de design trava esperando o backend criar mais um."*

"Não dá só pra paralelizar as chamadas, ou criar 1 endpoint novo?" — funciona pra **uma** tela.
O problema real é **N telas × M clientes × K serviços**: cada combinação nova vira mais 1 endpoint sob medida, mais 1 PR de backend, mais 1 contrato pra versionar. Paralelismo resolve latência de uma chamada; não resolve o front esperando dias por um endpoint novo toda sprint — isso é o que cresce sem controle, não a chamada isolada.

BFF — Backend-for-Frontend

O problema sem BFF

- RN, KMP, Web cada um bate em endpoints REST diferentes
- Over-fetching (campos que não usa) e under-fetching (N+1 requests)
- Múltiplos times sincronizando formatos de resposta

Com BFF + GraphQL

- Um endpoint `/graphql` serve **todos os clientes**
- Cliente pede **só os campos que precisa**
- Schema = contrato único versionado

```
# Android pergunta isso:
query {
  popularMovies {
    results { title voteAverage }
  }
}
```

```
# PWA pergunta isso:
query {
  popularMovies {
    results { title posterPath overview }
}
}
```

> **Mesma query shape, respostas diferentes.** O BFF otimiza por cliente sem mudar o backend.

GraphQL — paradigma alternativo

REST é orientado a **recursos** (`/movies/:id` , `/movies/:id/reviews`). GraphQL é orientado a **queries declarativas** — o cliente descreve o formato que quer, o servidor entrega exatamente isso.

```
query MovieDetail($id: ID!) {
  movie(id: $id) {
    id
    title
    director { id name }
    reviews(limit: 10) { id rating comment user { name } }
  }
}
```

> **Uma chamada**, exatamente os dados que o cliente precisa — nem mais, nem menos.

GraphQL — pontos fortes

- **Cliente decide** o shape do payload (sem over-fetch / under-fetch)
- **Schema strong-typed** — geração de tipos automática (TS/Kotlin) a partir do schema
- **Single endpoint** — uma URL, várias queries (REST tende a multiplicar endpoints)
- **Federation** — vários serviços expostos como 1 schema unificado (Netflix, Shopify usam pra isso)
- **Tooling forte** — Apollo Studio, GraphiQL/Sandbox, codegen

GraphQL — armadilhas (não é grátis)

> Mês 2 em produção: a query de `reviews` aninhada com `user` sem DataLoader disparou **500+ chamadas** no banco numa tela só. Servidor caiu. Isso é o N+1 batendo na cara — junto de outras 3 armadilhas que ninguém conta na demo do "Hello World":

❌ **N+1 query problem** — cada campo aninhado pode disparar 1 chamada nova no resolver

→ resolve com **DataLoader** (batching + caching por request)

❌ **Queries sem limite de profundidade** — cliente malicioso pede uma query gigante, agride o servidor

→ query depth limiting + rate limit por complexidade

❌ **Caching mais complexo que REST** — não é HTTP-cacheable de graça (CDN não ajuda igual)

→ normalização client-side (Apollo Cache) ou infra própria de cache

❌ **Curva de aprendizado** — schema design, resolvers, federation exigem time dedicado

GraphQL — quando faz sentido usar (e quando não)

Cresce em valor quando: múltiplos clientes (mobile/web/TV) com necessidades de dado diferentes do mesmo backend · muitos microsserviços atrás de 1 BFF (GraphQL agrega, cliente nem sabe que são 5 serviços) · time de front quer iterar sem esperar backend criar endpoint novo.

> É a resposta pro cenário dos **23 endpoints** de 2 slides atrás: 1 schema substitui a combinação tela×cliente inteira — endpoint novo não é mais um PR de backend, é uma query diferente.

Quem usa de verdade:

- **GitHub** — API pública migrou de REST (v3) pra GraphQL (v4) em 2016; hoje é a recomendada
- **Shopify** — Storefront API é GraphQL; mantém o `graphql-ruby` open source
- **Netflix** — GraphQL Federation (DGS, open source) agrega dezenas de times de backend num schema só
- **Facebook** — onde nasceu (2012, interno), motivado por app mobile lento com REST rígido

Time pequeno, poucos clientes? REST + BFF simples geralmente resolve com menos operação — GraphQL compensa quando a **complexidade de agregação** (muitos clientes/serviços) supera o custo de montar e manter schema + resolvers. Não é "melhor sempre" — é trocar complexidade de lugar, não eliminar ela.

TMDB REST vs GraphQL — dois sabores do mesmo problema

	TMDB REST (exercícios)	CineHub GraphQL (demo na próxima aula)
Autenticação	Bearer token no header	JWT local (register/login)
Popular movies	`GET /movie/popular`	`query { popularMovies }`
Busca	`GET /search/movie?query=x`	`query { searchMovies(query:"x") }`
Over-fetching	Sim (resposta fixa)	Não (cliente escolhe campos)
Offline	Cache HTTP / SW	Normalização client-side

No KMP usamos Ktor + TMDB REST — simples, sem servidor local.

Na PWA usamos fetch + TMDB REST + Service Worker para offline.

No BFF real (capstone) tudo passa pelo GraphQL.

> **Lição do Bloco 2:** GraphQL não substitui REST — troca complexidade de lugar, só compensa quando agregação multi-cliente supera o custo do schema (ver "quando faz sentido usar", alguns slides atrás).

O que já vem pronto pra Atividade 5: Movie + TmdbApi

```
// shared/.../data/Movie.kt — o dado que a UI vai receber
data class Movie(
    val id: Int, val title: String, val overview: String,
    val posterPath: String?, val releaseDate: String, val voteAverage: Double,
)

// shared/.../data/TmdbApi.kt — já implementado, busca de verdade via Ktor
class TmdbApi(token: String) {
    suspend fun popularMovies(page: Int = 1): MoviesResponse
}
```

> Vocês não tocam nesses dois arquivos — é o "prato pronto". A pergunta do exercício é: como transformar um `List<Movie>` numa tela?

Compose — os blocos que o exercício usa

Bloco	Pra quê	Usado em
`when { ... }`	escolher a tela certa (token ausente / carregando / erro / lista)	TODO 1
`LazyColumn { item {...}; items(lista) {...} }`	lista rolável com cabeçalho + itens	TODO 2
`Card`, `Row`, `Column`	compor um item da lista (poster + texto)	TODO 3
`Text(..., maxLines=2, overflow=TextOverflow.Ellipsis)`	truncar texto longo sem quebrar o layout	TODO 3

```
// mini-exemplo isolado — não é o exercício, é só pra ver o bloco funcionando
LazyColumn {
    item { Text("Cabeçalho") }
    items(listOf("A", "B", "C")) { nome -> Text(nome) }
}
```

> Todo TODO da Atividade 5 é montar essas peças com o dado real do `Movie` — nada disso é novo depois daqui, só ligar API + Compose.

Compose — componentes principais

```
@Composable                                    // toda UI é uma função marcada assim
fun Ola(nome: String) {
    var contador by remember { mutableStateOf(0) }   // state local — muda, tela redesenha

    Column(modifier = Modifier.padding(16.dp)) {      // empilha verticalmente
        Text("Olá, $nome! ($contador)")                // texto
        Row {                                           // empilha horizontalmente
            Button(onClick = { contador++ }) { Text("+1") }   // clicável
        }
        Box(contentAlignment = Alignment.Center) { /* sobrepõe elementos */ }
    }
}
```

Bloco	Empilha/faz
`Column`	vertical
`Row`	horizontal

> `@Composable` = função de UI · `remember { mutableStateOf(...) }` = state · `Modifier` sempre encadeado. São as peças que vocês já usaram sem perceber no App.kt (`Surface`, `MaterialTheme`).

🌐 Hands-on 2 — construa um MovieCard no @Preview

Diferente do Kotlin Playground (só lógica), Compose não roda no navegador — usamos o **@Preview** do Android Studio: atualiza a tela sem precisar de emulador.

```
@Preview
@Composable
fun CardPreview() {
    MaterialTheme {
        /* 👈 suas peças entram aqui */
        Text("comece aqui")
    }
}
```

> A cada peça: troca o `/* suas peças entram aqui */` → **Build** (ou ⌘+' / Ctrl+Alt+Shift+U) no painel Preview. Sem emulador, sem app rodando.

Peça 1 · `Text` (o título)

`Text` mostra uma string. O visual vem de parâmetros: `fontSize`, `fontWeight`, `color`.

```
Text("Matrix", fontWeight = FontWeight.Bold, fontSize = 22.sp)
```

> 👨‍💻 **Sua vez:** ponha o título **Matrix** no lugar do `/* suas peças entram aqui */` e dê Build. Experimente: mude o `fontSize` e adicione `color = Color.Blue` — veja a tela reagir.

Peça 2 · `Row` (a nota)

`Row { ... }` coloca composables **lado a lado** (horizontal).

```
Row {
    Text("★")
    Text(" 8.7")
}
```

> 👨‍💻 **Sua vez:** monte a linha da nota ⭐ **8.7** — dois `Text` dentro de um `Row`.

Peça 3 · `Column` (empilhar)

`Column` é o `Row` na **vertical**. Empilha os composables filhos de cima pra baixo.

```
Column {
    /* título · nota · ano */
}
```

> 👨‍💻 **Sua vez:** empilhe num `Column`: o **título** (peça 1) + a **linha da nota** (peça 2) + o **ano** → `Text("1999", color = Color.Gray)`.

Peça final · `Card` + `Modifier.padding` = MovieCard 🎉

`Card` dá a moldura com elevação; `Modifier.padding(16.dp)` dá o respiro interno. Embrulhe o `Column`:

```
Card {
    Column(modifier = Modifier.padding(16.dp)) {
        Text("Matrix", fontWeight = FontWeight.Bold, fontSize = 22.sp)
        Row { Text("★ 8.7") }
        Text("1999", color = Color.Gray)
    }
}
```

> 🎉 Vocês **construíram** um MovieCard — não copiaram. É a mesma peça que vocês montam de verdade no **TODO 3 da Atividade 5** (só troca texto fixo pelo `movie.title` / `movie.overview`).

Atividade 5 — KMP: filmes populares (10 pts)

Stack: o mesmo KotlinProject do Bloco 1 (Android/iOS/Web) + Ktor + TMDB REST API

Já pronto (`shared/src/commonMain/kotlin/.../data/`): `Movie.kt` + `TmdbApi.kt` (busca real via Ktor). **O exercício é construir a UI** em `App.kt` — 3 TODOs:

```
TODO 1  estados: token ausente / carregando / erro / lista      (bloco: when)
TODO 2  MovieList — LazyColumn com cabeçalho + itens              (bloco: LazyColumn)
TODO 3  MovieCard — poster placeholder + título + overview + nota/ano  (bloco: Card/Row/Column)
```

> Blocos de Compose de cada TODO: ver slide "Compose — os blocos que o exercício usa", 2 antes.

Setup:

```
cd exercicios/05-kmp/pratica
cp local.properties.example local.properties
# adicione seu tmdb.token= no local.properties
# Sync Gradle → Run androidApp
```

Entrega: fork → PR tocando `exercicios/05-kmp/pratica/` · Prazo: ver Canvas

Projeto Final — tema livre, app bonito e bem arquitetado

O Projeto Final é um projeto à parte (BFF/GraphQL foi só conceito). O grupo constrói **um app mobile** em **Flutter, React Native ou KMP** e entrega por **PR**.

- **Opção A:** um dos **5 temas prontos** (PokeAPI, Rick and Morty, TheMealDB, TheCocktailDB, Studio Ghibli) com **skeleton** de ponto de partida
- **Opção B: qualquer tema que o grupo quiser** (hábitos, clima, cinema, estudo, viagem…) — do zero

Mesma rubrica nas duas, **sem bônus** por escolher uma ou outra. O skeleton tem visual básico de propósito: a nota de UI/UX é do **seu** trabalho. **Não há autograder** — eu clono o PR, buildo e avalio pela rubrica.

Enunciado completo: `exercicios/projeto-final/enunciado.md`

Projeto Final — logística

Grupo: 3 a 4 pessoas, auto-organizados — combinem entre vocês hoje/nesta semana.

Entrega: PR do **fork do grupo** → repo da disciplina — prazo: **ver Canvas** (**2 semanas** a partir de hoje).

Apresentação: ao vivo, demo funcionando — **10min + 5min de Q&A**, na **Aula 6** (ver Canvas).

Registrem por issue no repo público: `Grupo <nomes> — <tema> + <framework>`.
Opção A: cada tema+framework de skeleton é de **um grupo só** (ordem de chegada). Opção B: sem restrição.

> Formem o grupo **hoje** — tema e framework são decisão de grupo, não individual.

Projeto Final — 2 semanas, 4 marcos

Quando	Marco	Como o grupo comprova
Hoje / até o fim da semana 1	Grupo formado + tema e framework escolhidos	**Issue** aberta
Meio do caminho (checkpoint, sem nota)	App builda, lista + detalhe funcionando	**PR final aberto** (pode ser *draft*)
Semana 2	Persistência, estados de erro, checklist de UI/UX, README + ADR	PR final atualizado; **ensaiem os 10 min** da demo
Aula 6	Apresentação ao vivo (10 min + 5 Q&A)	App rodando no emulador/celular

> Sem apresentação intermediária: o **PR aberto cedo** é o checkpoint — eu olho o PR e aviso se algo está fora do esperado. Quem deixa o PR pra última noite perde a chance de corrigir o build (**build quebrado = Funcionalidade zerada**).

Projeto Final — como trabalhar em grupo (1 fork por grupo)

Passo	Quem	O quê
1	**1 integrante**	Faz o **fork** do repo da disciplina → vira o **repo do grupo** (adiciona os colegas como *collaborators*)
2	**Cada integrante**	Clona o **fork do grupo**, cria branch com a sua parte
3	**Cada integrante**	Abre **PR interno → `main` do fork do grupo**; o grupo revisa e mergeia
4	**O grupo**	Abre o **PR final**: `main` do fork → repo da disciplina

Onde fica o app: tema livre → `exercicios/projeto-final/entregas/<nome-do-grupo>/` · skeleton → `skeletons/<tema>/<framework>/pratica/` (comecem pelo `README.md` dele).

Por que PRs internos? Contribuição individual (3 pts) olha os commits/PRs de cada conta — tudo commitado por uma pessoa só = grupo perde esses pontos.

Projeto Final — o que o app precisa ter

#	Requisito	Detalhe
1	Lista de dados reais	Busca de uma **API/fonte real** (não lista hardcoded)
2	Detalhe	Tocar num item abre mais informação
3	Busca ou filtro	Feito pelo usuário
4	Persistência local	Favoritos/histórico/preferências **sobrevivem a fechar o app**
5	Estados de erro	**Carregando, vazio e sem internet/erro** com mensagem — nunca tela branca

> Tema livre: API/dados **gratuitos** (sem cartão), **nunca commitar chave de API**.

Projeto Final — UI/UX: o mínimo pra ficar bonito

- **Design tokens** — cores, tipografia, espaçamento **em um lugar só** (tema), não valores soltos
- **Tema claro e escuro** funcionando
- **Loading com skeleton/shimmer**, não só spinner solto
- **Imagens** com placeholder e fallback
- **Responsivo** — celular e paisagem/tela maior sem quebrar
- **Acessibilidade básica** — contraste, alvo de toque ≥ 48dp, ícones com descrição
- **Identidade** — **ícone e splash próprios**; navegação consistente
- **Sem cara de protótipo** — hierarquia visual, componentes reutilizáveis

Prova: pelo menos **3 screenshots no README** + a demo ao vivo.

Projeto Final — arquitetura, ADR e testes

- **Camadas:** dados (API/storage) · estado/lógica · UI — **sem fetch dentro do widget/componente**
- **Gerenciamento de estado** coerente com o framework
- **`ADR.md` (~1 página):** por que esse framework e esse gerenciador de estado, **que alternativa descartaram e por quê** (formato da Atividade 1)
- **Testes próprios** (unit/widget, ≥ 3 casos) para a lógica que vocês criaram — **não existe teste "pra passar"**; vale o que decidiram testar
- **README** reprodutível: como rodar (com o `cd`), screenshots, link do ADR

Projeto Final — rubrica (30 pts)

Critério	Pts
UI/UX — checklist + demo	8
Arquitetura + ADR	7
Funcionalidade — 5 requisitos, estados de erro, build passando	6
Apresentação — demo ao vivo + Q&A	4
Contribuição individual	3
Qualidade — testes próprios + README	2

> Sem autograder: nota lançada no Canvas depois da apresentação. **App que não builda = Funcionalidade zerada.**

Projeto Final — Opção A: temas prontos (skeletons)

#	Tema	API retorna	Filtro natural	Tipo do id
1	PokeAPI — Pokédex (151)	nome, sprite, tipos, altura/peso	`type`	`Int`
2	Rick and Morty — personagens (826)	nome, status, espécie, gênero	`status`	`Int`
3	TheMealDB — receitas (25)	nome, categoria, área, foto	`c`	`String`
4	TheCocktailDB — drinks (100)	nome, categoria, copo, alcoólico	`c`	`String`
5	Studio Ghibli — filmes (22)	título, diretor, ano, duração	`director`	`String` (UUID)

> O skeleton já estrutura lista/detalhe/busca/filtro/favoritos com `// TODO`, mas **persistência, estados de erro e o checklist de UI/UX continuam sendo do grupo**. Os flows Maestro vêm de brinde pra autoverificação — **não valem ponto**. Detalhes: `exercicios/projeto-final/temas/`.

Leituras complementares

- **JetBrains** — *Kotlin Multiplatform Docs* (kotlinlang.org/docs/multiplatform)
- **Ktor** — *ktor.io/docs/getting-started-ktor-client.html*
- **Newman, S.** — *Building Microservices* 2ed, capítulo 4 (BFF)
- **Porcello & Banks** — *Learning GraphQL* (O'Reilly)

Obrigado!

📧 jackson.96@gmail.com
🔗 linkedin.com/in/3jacksonsmith
🐙 github.com/jacksonsmith

Bom trabalho final! 🚀